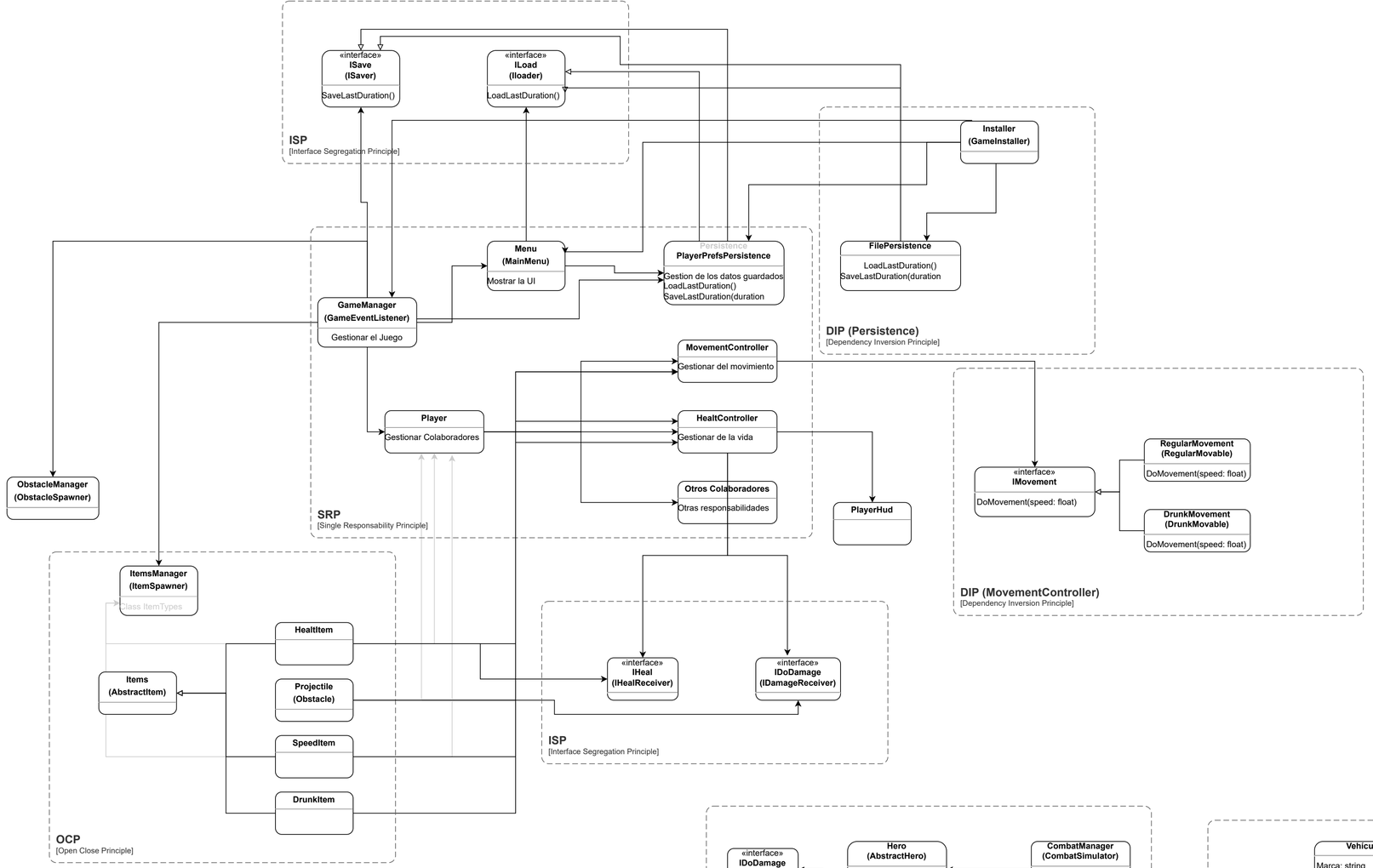
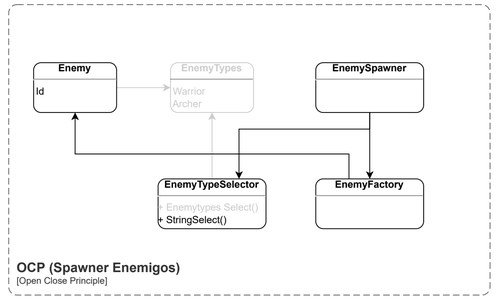
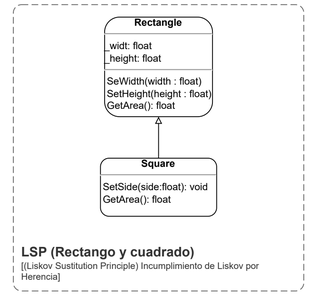
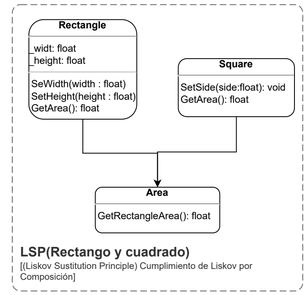
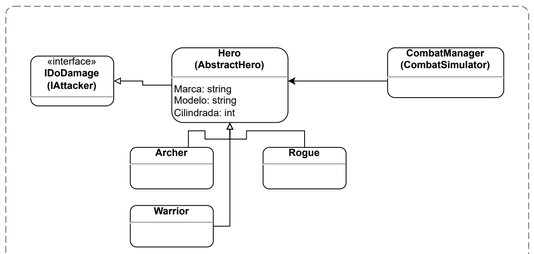
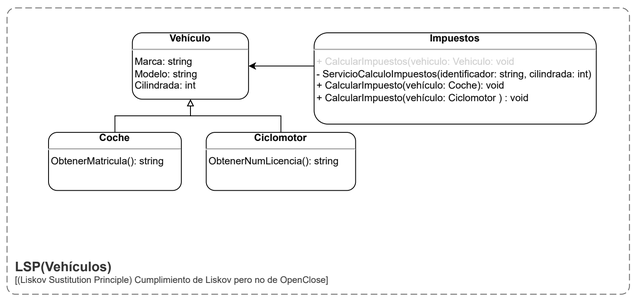

«interface»
ISave
(ISaver)
SaveLastDuration()
«interface»
ILoad
(Iloader)
LoadLastDuration()
ISP
[Interface Segregation Principle]
Installer
(GameInstaller)
Menu
(MainMenu)
Mostrar la UI
Persistence
PlayerPrefsPersistence
Gestion de los datos guardados
LoadLastDuration()
SaveLastDuration(duration
FilePersistence
LoadLastDuration()
SaveLastDuration(duration
DIP (Persistence)
[Dependency Inversion Principle]
GameManager
(GameEventListener)
Gestionar el Juego
MovementController
Gestionar del movimiento
Player
Gestionar Colaboradores
HealtController
Gestionar de la vida
Otros Colaboradores
Otras responsabilidades
PlayerHud
SRP
[Single Responsability Principle]
ObstacleManager
(ObstacleSpawner)
«interface»
IMovement
DoMovement(speed: float)
RegularMovement
(RegularMovable)
DoMovement(speed: float)
DrunkMovement
(DrunkMovable)
DoMovement(speed: float)
DIP (MovementController)
[Dependency Inversion Principle]
ItemsManager
(ItemSpawner)
Class ItemTypes
HealtItem
Items
(AbstractItem)
Projectile
(Obstacle)
SpeedItem
DrunkItem
«interface»
IHeal
(IHealReceiver)
«interface»
IDoDamage
(IDamageReceiver)
ISP
[Interface Segregation Principle]
OCP
[Open Close Principle]

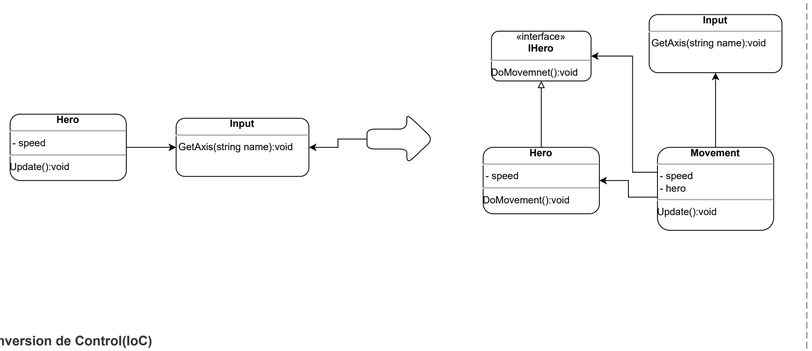

Enemy
Id
EnemyTypes
Warrior
Archer
EnemySpawner
EnemyTypeSelector
+ EnemyTypes Select()
+ StringSelect()
EnemyFactory
OCP (Spawner Enemigos)
[Open Close Principle]


Rectangle
_widt: float
_height: float
SeWidth(width : float)
SetHeight(height : float)
GetArea(): float
Square
SetSide(side:float): void
GetArea(): float
LSP (Rectango y cuadrado)
[(Liskov Sustitution Principle) Incumplimiento de Liskov por Herencia]


Rectangle
_widt: float
_height: float
SeWidth(width : float)
SetHeight(height : float)
GetArea(): float
Square
SetSide(side:float): void
GetArea(): float
Area
GetRectangleArea(): float
LSP(Rectango y cuadrado)
[(Liskov Sustitution Principle) Cumplimiento de Liskov por Composición]


«interface»
IDoDamage
(IAttacker)
Hero
(AbstractHero)
Marca: string
Modelo: string
Cilindrada: int
CombatManager
(CombatSimulator)
Archer
Rogue
Warrior
LSP(Sistema de combate)
[(Liskov Sustitution Principle) Resuelve Liskov a traves de propiedades booleanas pero se espera una mejor solucion aplicando Inversion de Dependencias mas adelante]


Vehículo
Marca: string
Modelo: string
Cilindrada: int
Impuestos
+ CalcularImpuestos(vehículo: Vehículo: void
- ServicioCalculoImpuestos(identificador: string, cilindrada: int)
+ CalcularImpuesto(vehículo: Coche): void
+ CalcularImpuesto(vehículo: Ciclomotor) : void
Coche
ObtenerMatricula(): string
Ciclomotor
ObtenerNumLicencia(): string
LSP(Vehículos)
[(Liskov Sustitution Principle) Cumplimiento de Liskov pero no de OpenClose]


Hero
- speed
Update():void
Input
GetAxis(string name):void
«interface»
IHero
DoMovemnet():void
Input
GetAxis(string name):void
Hero
- speed
DoMovement():void
Movement
- speed
- hero
Update():void
Inversion de Control(IoC)
[No forma parte de Solid, se basa en que si tenemos dos clases en lugar de que la primera controle la segunda, lo que hacemos es inyectar la primera en la segunda y la segunda será la que controle a la primera, Tambien se le conoce como principio de Holliwood que dice "No nos llames, nosotros te llamaremos"]